Classification

Session 4

Yuhua Li
liy180@cardiff.ac.uk

Contents

- *k*-nearest neighbours
- Support Vector Machines
- Decision trees
- Hyperparameter Optimisation

Classification: Definition

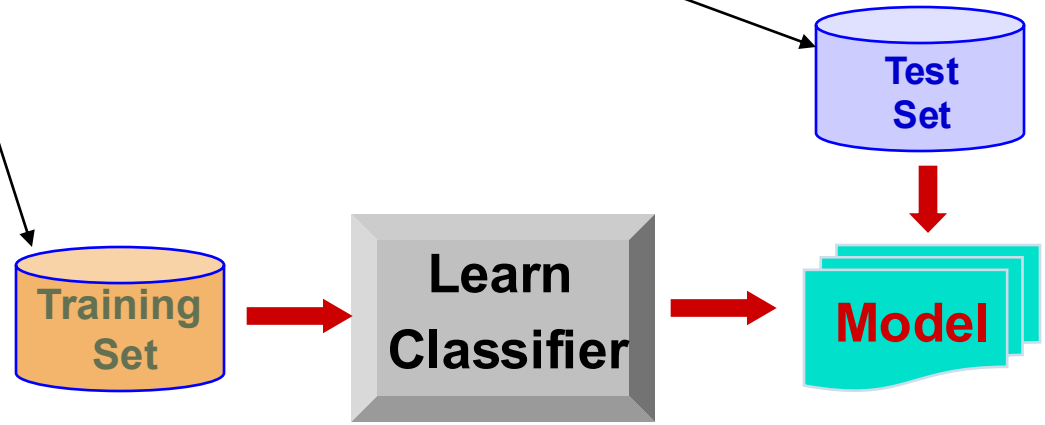
- Given a collection of records (*training set*)
  - Each record contains a set of *attributes*, one of the attributes is the *class*.
- Find a *model* for class attribute as a function of the values of other attributes.
- Goal: previously unseen records should be assigned a class as accurately as possible.
  - A *test set* is used to determine the accuracy of the model. Usually, the given data set is divided into training and test sets, with training set used to build the model and test set used to evaluate it.

Classification Example

categorical categorical continuous class

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Refund	Marital Status	Taxable Income	Cheat
No	Single	75K	?
Yes	Married	50K	?
No	Married	150K	?
Yes	Divorced	90K	?
No	Single	40K	?
No	Married	80K	?



Contents

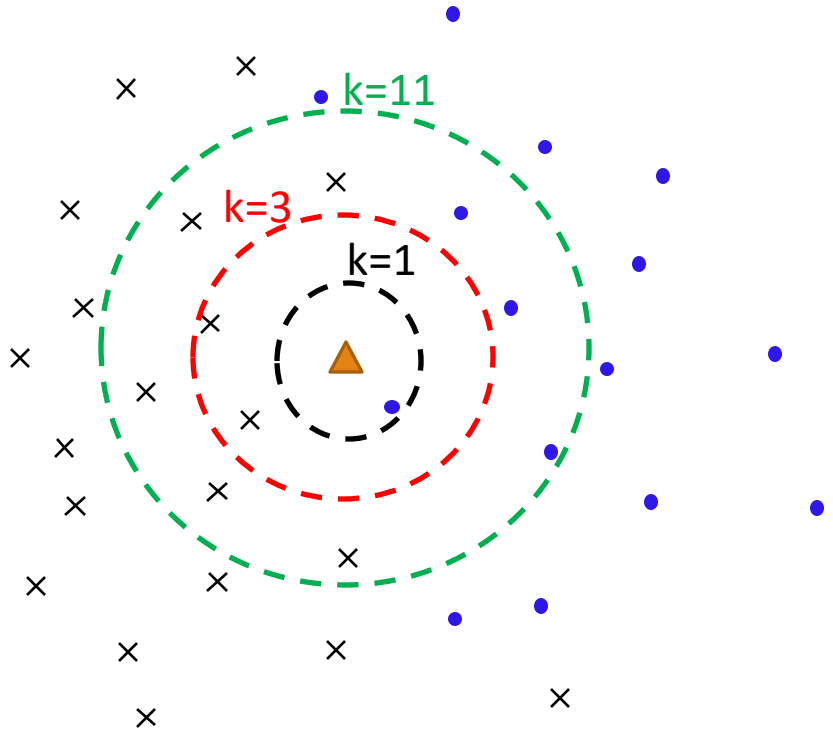
- ***k*-nearest neighbours**
- Support Vector Machines
- Decision trees
- Hyperparameter Optimisation

k-nearest neighbours

- A type of instance-based learning, or lazy learning
- A nonparametric method
- Can be used for both classification and regression, though commonly used for classification.
- Learning is very fast (finding *k* or no training)
  - Store training examples

k-NN Classification

- Training Algorithm
  - Store training data points
- Set the value for ***k***
- Prediction Algorithm
  - To classify a new input vector ***x***:
    - Calculate the distance from ***x*** to each of the data points in training set
    - Find the k training data points which are nearest to ***x***
    - Examine the ***k*-nearest** training data points to ***x*** and assign ***x*** to the most frequently occurring class
- A commonly used distance metric for continuous variables is **Euclidean distance**



k-NN Classification

- Advantages
  - Training is very fast (no training required)
  - Learn complex target functions
  - Simple and intuitive, easy to program
  - Classification accuracy can be very good; can outperform more complex models
- Disadvantages
  - Slow at query time
  - Sensitive to noise and irrelevant attributes

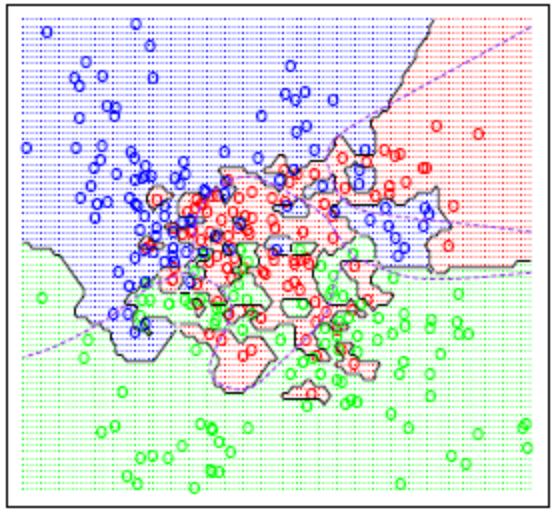
k-NN Issues

- Distance measure
  - Most common: Euclidean distance

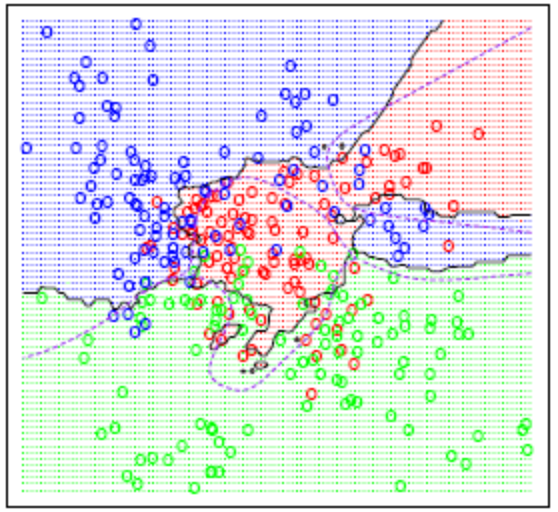
$$d(\mathbf{x}_i, \mathbf{x}_j) = \left(\sum_{l=1}^{n} (x_{i,l} - x_{j,l})^2 \right)^{1/2}$$

- Choosing *k*
  - Increasing *k* reduces variance, increases bias
- Memory-based technique. Must make a pass through the data for each classification. This can be prohibitive for large data sets.
- Feature scaling is needed if features are not commensurate

K=1



K=15



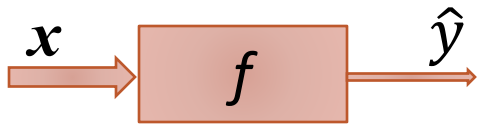
Contents

- K nearest neighbours
- **Support Vector Machines**
- Hyperparameter Optimisation
- Decision trees

Support Vector Machines

$f(\boldsymbol{x}) = 1.5\, x_1 - x_2 - 0.1$

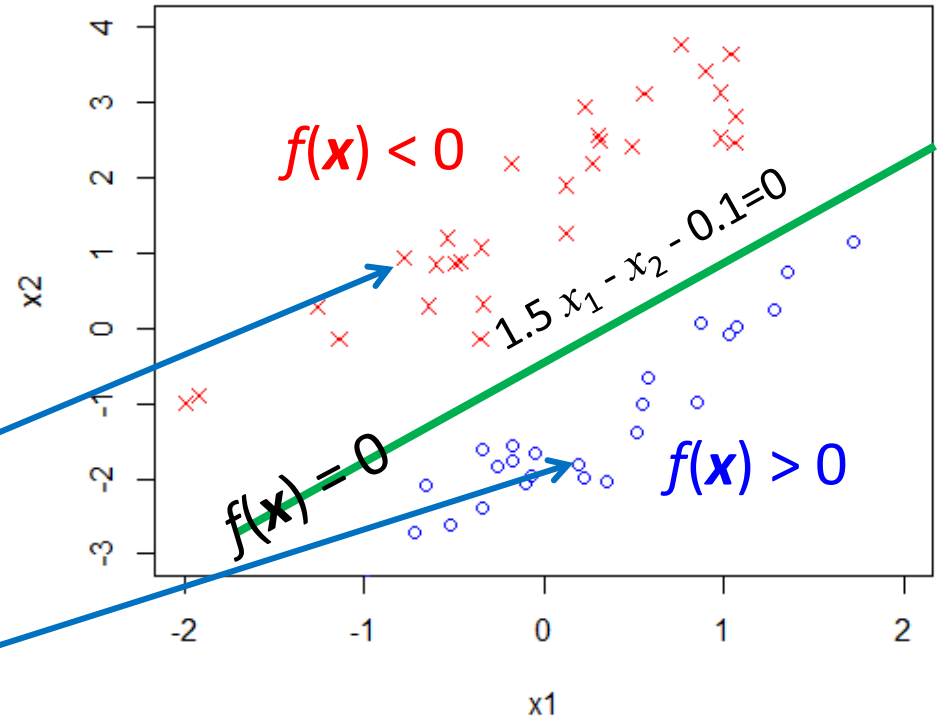
- Linear classifiers for binary classes



$$f(\boldsymbol{x},w,b)=sign(\boldsymbol{w}\cdot\boldsymbol{x}+b)$$

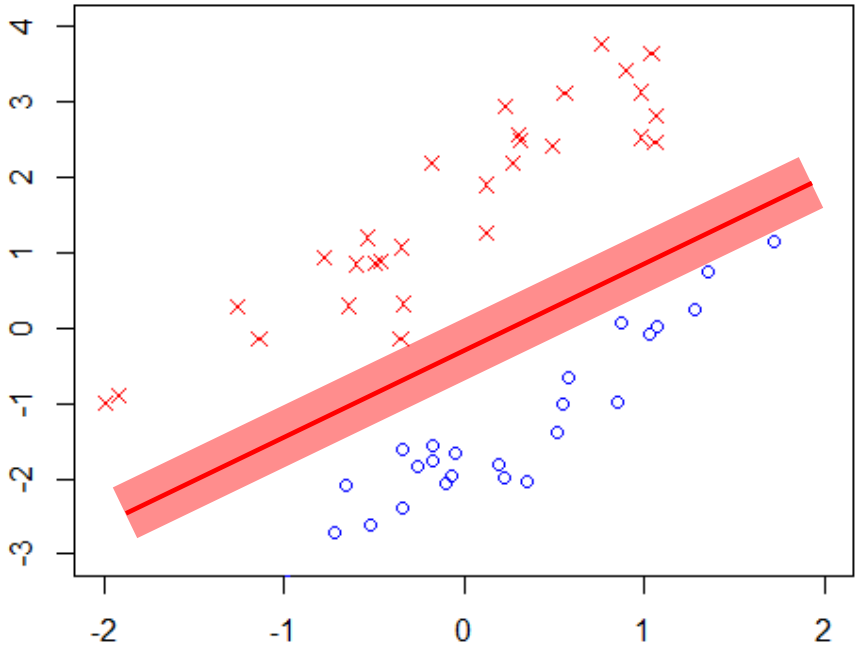
$$1.5x_1 - x_2 - 0.1 = 0$$

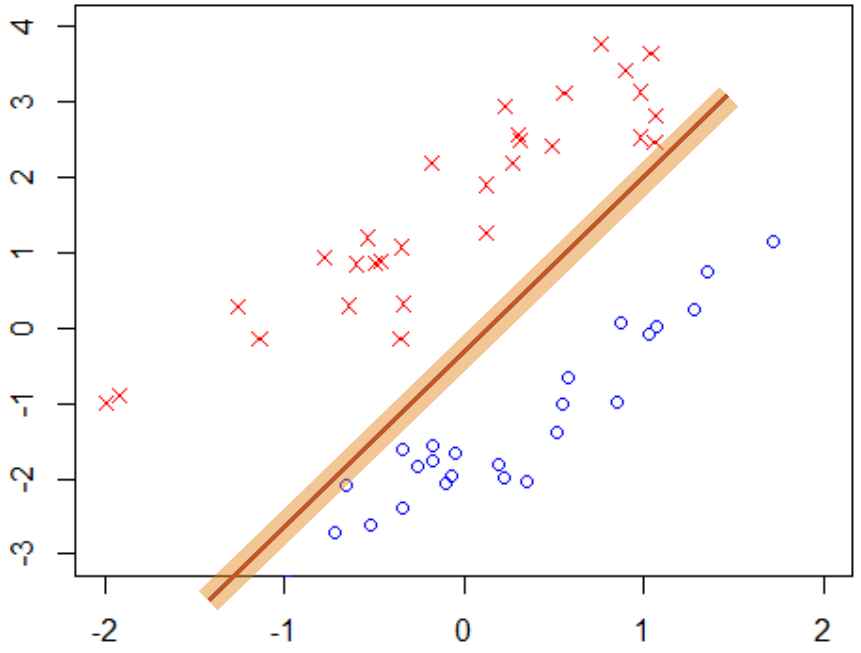
$\boldsymbol{w}$=(1.5, -1), $\boldsymbol{b}$=−0.1

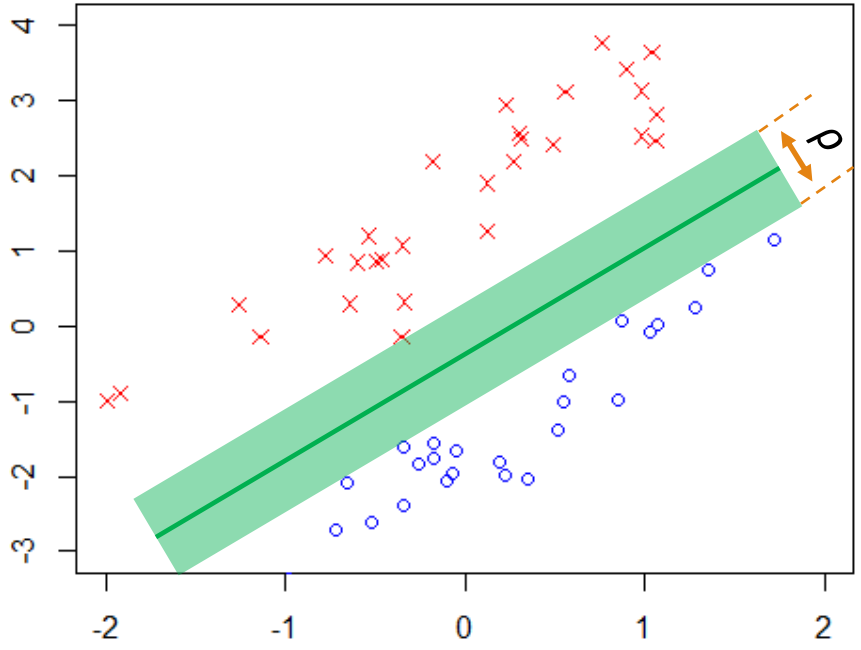


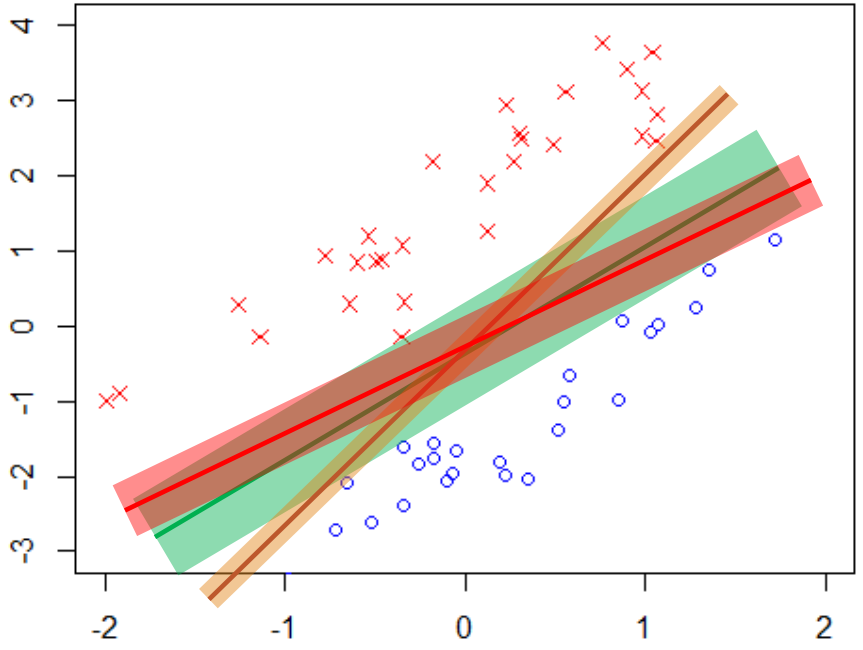
For a point (-0.8, 1), *sign*(1.5*(-0.8) - 1 – 0.1) = *sign*(-2.3) = -1

For a point (0.1, -2), *sign*(1.5*0.1 - (-2) – 0.1) = *sign*(2.05) = +1


4
3
2
1
0
-1
-2
-3
-2
-1
0
1
2


4
3
2
1
0
-1
-2
-3
-2
-1
0
1
2


d
4
3
2
1
0
-1
-2
-3
-2
-1
0
1
2


4
3
2
1
0
-1
-2
-3
-2
-1
0
1
2

Support Vector Machines

- As the 2-d example shown on previous slide, the two classes can be separated by different lines.
- The points nearest to the separation/decision line (or plane in 3-d or hyperplane in higher dimension) define the margin(i.e., gap) between classes.
- Those points are called support vectors.
- Support vector machine aims to maximise the margin.

Maximum Margin: Formalization

- $\boldsymbol{w}$: decision hyperplane normal vector
- $\boldsymbol{x_i}$: data point i
- y_i: class of data point i (+1 or -1) NB: Not 1/0
- Classifier is: $f(\boldsymbol{x}_i) = sign(\boldsymbol{w}^T \boldsymbol{x}_i + b)$
- Functional margin of $\boldsymbol{x}_i$ is: $y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b)$
  - But note that we can increase this margin simply by scaling ***w***, ***b***….
- Functional margin of dataset is **twice** the minimum functional margin for any point
  - The factor of 2 comes from measuring the whole width of the margin

Linear Support Vector Machine (SVM)

- **Hyperplane**

$$\mathbf{w}^T \mathbf{x} + b = 0$$

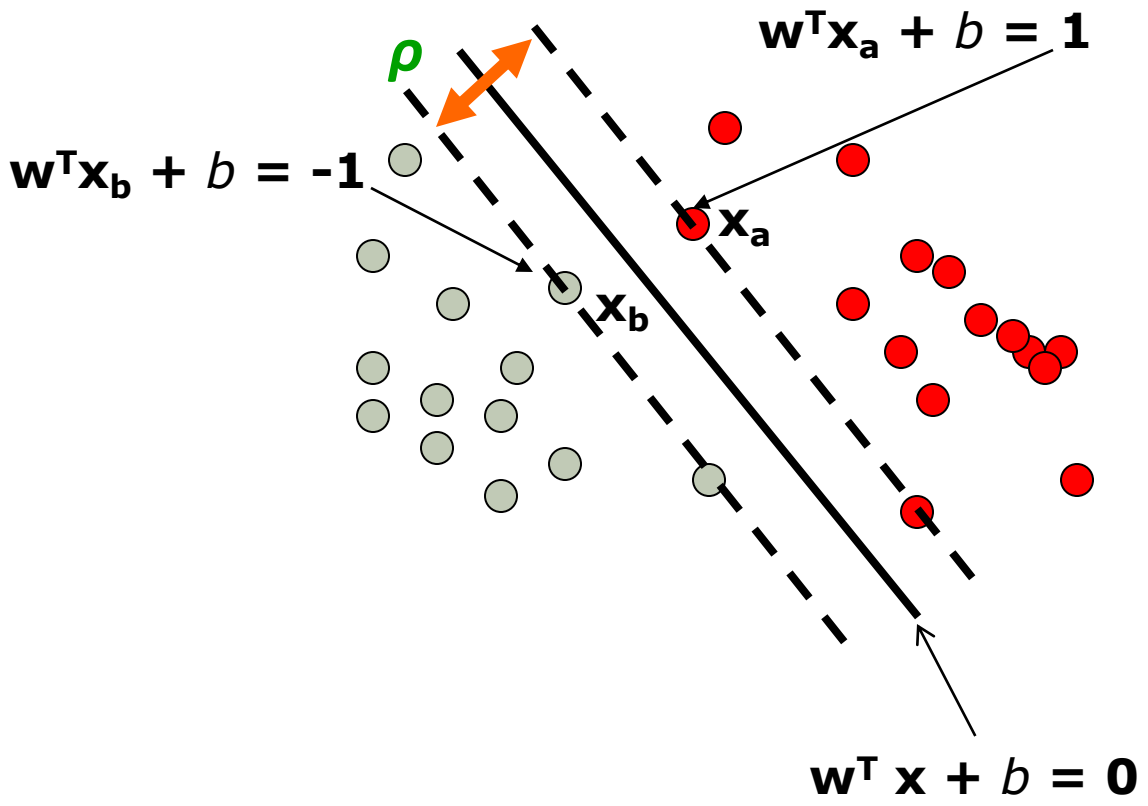
- **Extra scale constraint**:

$$min_{i=1,\ldots,N} |\boldsymbol{w}^T \mathbf{x}_i + 1| = 1$$

- This implies:

$$\boldsymbol{w}^T(\mathbf{x}_a - \mathbf{x}_b) = 2$$

$$\boldsymbol{\rho} = \|\mathbf{x}_a - \mathbf{x}_b\|_2 = \frac{2}{\|\boldsymbol{w}\|_2}$$



Linear SVMs Mathematically (cont.)

- Then we can formulate the *quadratic optimization problem:*

Find $\mathbf{w}$ and b such that

$$\rho = \frac{2}{\|\mathbf{w}\|} \text{ is maximized; and for all } \{(\mathbf{x_i}, y_i)\}$$

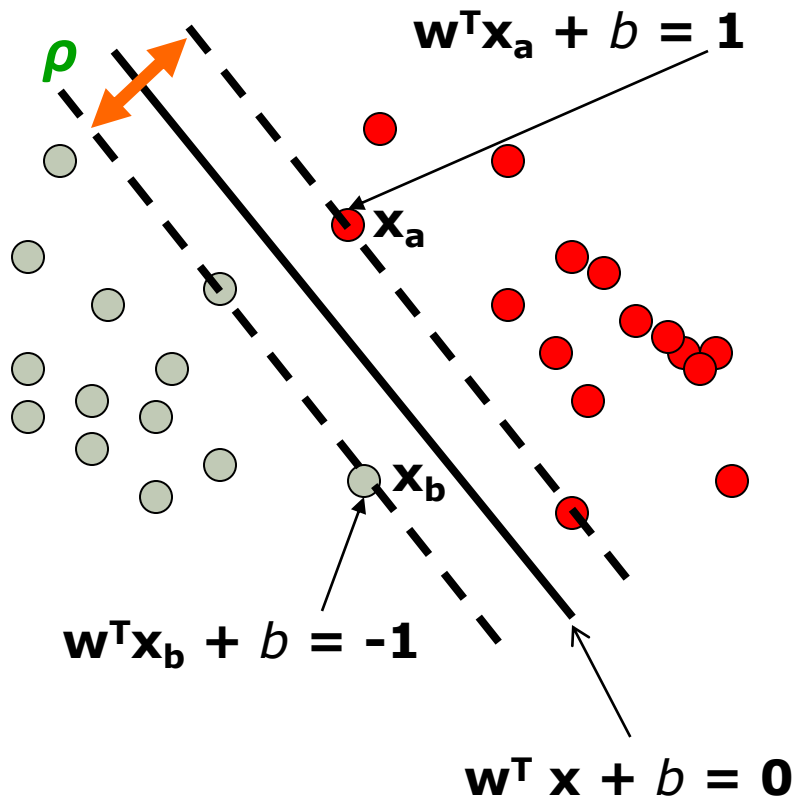
$$\mathbf{w^T x_i} + b \geq 1 \text{ if } y_i = 1; \quad \mathbf{w^T x_i} + b \leq -1 \text{ if } y_i = -1$$

- A better formulation (min **||w||** = max 1/ **||w||**):

Find $\mathbf{w}$ and b such that

$$L(\mathbf{w}) = \tfrac{1}{2}\, \mathbf{w^T w} \text{ is minimized;}$$

$$\text{and for all } \{(\mathbf{x_i}, y_i)\}: \quad y_i\, (\mathbf{w^T x_i} + b) \geq 1$$



The Optimization Problem Solution

- The solution has the form:

$$\mathbf{w} = \Sigma \alpha_i y_i \mathbf{x_i}$$
$$b = y_k - \mathbf{w}^T \mathbf{x}_k \text{ for any } \mathbf{x}_k \text{ such that } \alpha_k \neq 0$$

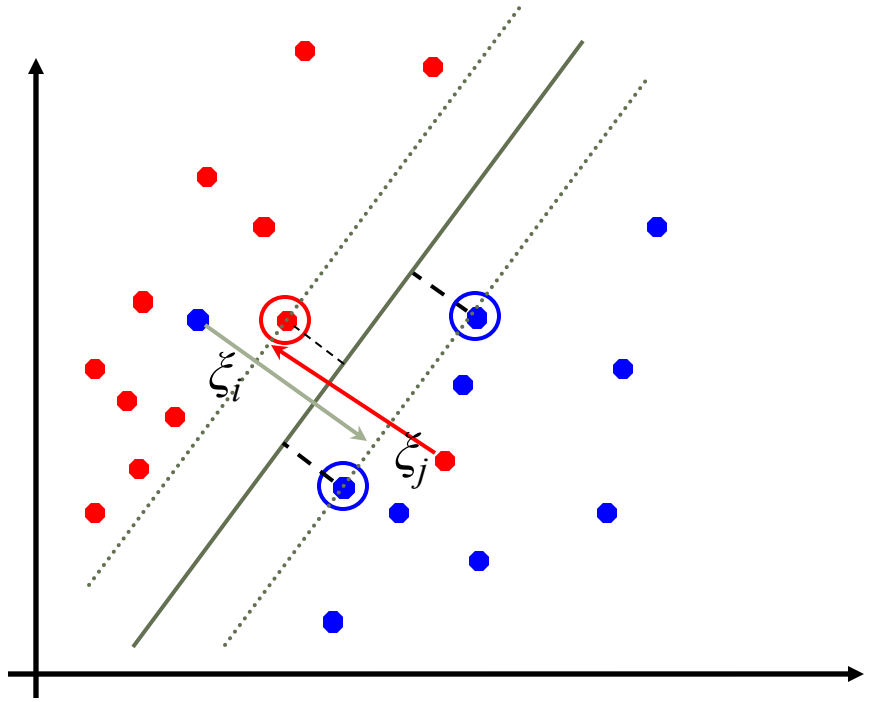
- Each non-zero α_i indicates that corresponding $\mathbf{x}_i$ is a support vector.
- Then the classifying function will have the form:

$$f(\mathbf{x}) = \Sigma \alpha_i y_i \mathbf{x}_i^T \mathbf{x} + b$$

- Notice that it relies on an *inner product* between the test point $\mathbf{x}$ and the support vectors $\mathbf{x_i}$
  - We will return to this later.
- Also keep in mind that solving the optimization problem involved computing the inner products $\mathbf{x}_i^T \mathbf{x}_j$ between all pairs of training points.

Soft Margin Classification

- If the training data is not linearly separable, *slack variables* ξ_i can be added to allow misclassification of difficult or noisy examples.
- Allow some errors
  - Let some points be moved to where they belong, at a cost
- Still, try to minimize training set errors, and to place hyperplane "far" from each class (large margin)



Soft Margin Classification Mathematically

- The old formulation:

Find $\mathbf{w}$ and b such that
$L(\mathbf{w}) = \frac{1}{2}\,\mathbf{w}^T\mathbf{w}$ is minimized and for all $\{(\mathbf{x}_i, y_i)\}$
$y_i\,(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$

- The new formulation incorporating slack variables:

Find $\mathbf{w}$ and b such that
$L(\mathbf{w}) = \frac{1}{2}\,\mathbf{w}^T\mathbf{w} + C\,\Sigma\xi_i$ is minimized and for all $\{(\mathbf{x}_i, y_i)\}$
$y_i\,(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i$ and $\xi_i \geq 0$ for all i

- Parameter C can be viewed as a way to control overfitting
  - A regularization term

Regularisation

- Support vector machines
  - Training SVM involves regularization by default, its strength of regularization is determined by the penalty parameter $C>0$.

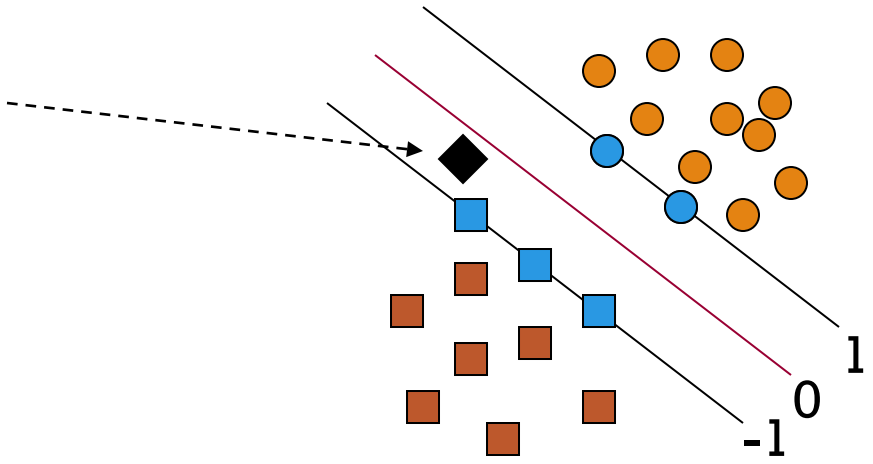
$$\min \quad \frac{1}{2} w^T w + C \sum_{i=1}^{N} \xi_i$$

  - Larger values of *C*: less regularization
    - Fit the training data as well/accurate as possible
    - Each individual data point is important to classify correctly
  - Smaller values of *C*: more regularization
    - More tolerant of errors on individual data points

Classification with SVMs

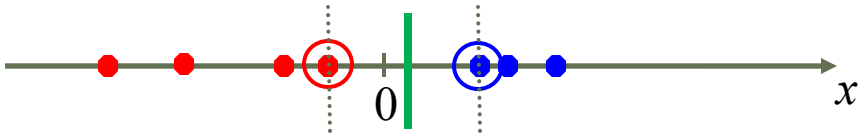
- Given a new point **x**, we can score its projection onto the hyperplane normal:
  - *i.e.*, compute score: $\mathbf{w}^T\mathbf{x} + b = \Sigma \alpha_i y_i \mathbf{x}_i^T\mathbf{x} + b$
    - Decide the class based on whether the score <0 or >0
  - Can set confidence threshold *t*.

Score > *t*:	yes
Score < -*t*:	no
Else:	don't know



Non-linear SVMs

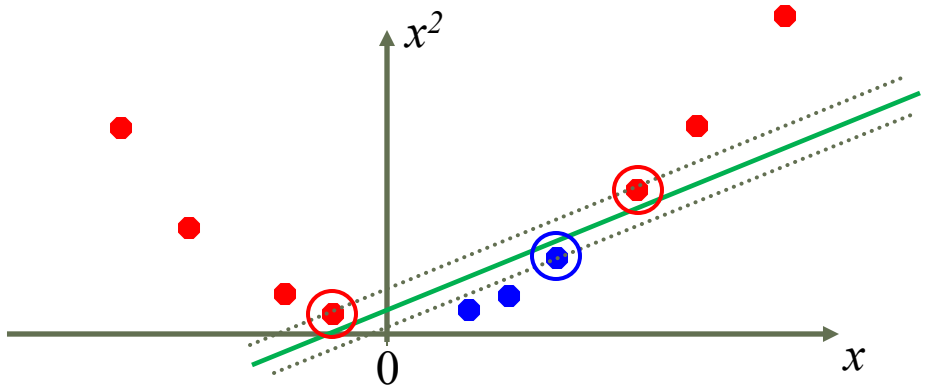
- Datasets that are linearly separable (with some noise) work out great:

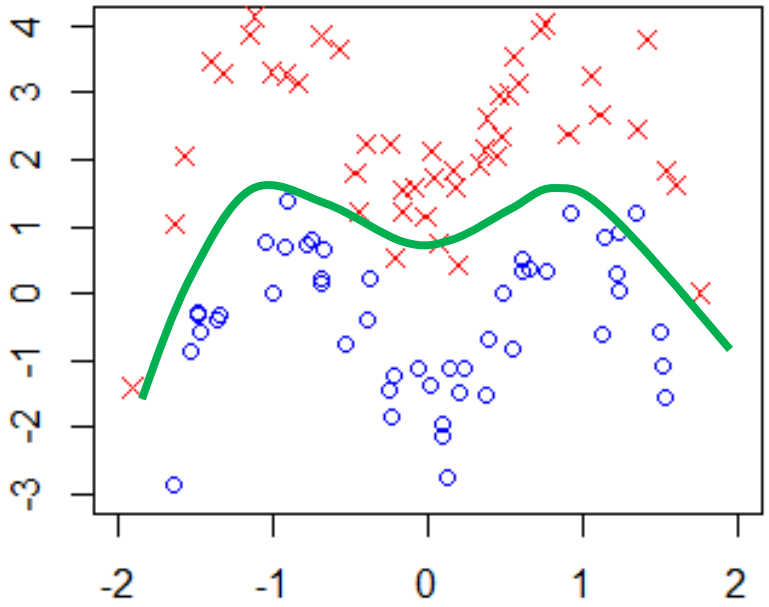


- But what are we going to do if the dataset is just too hard?



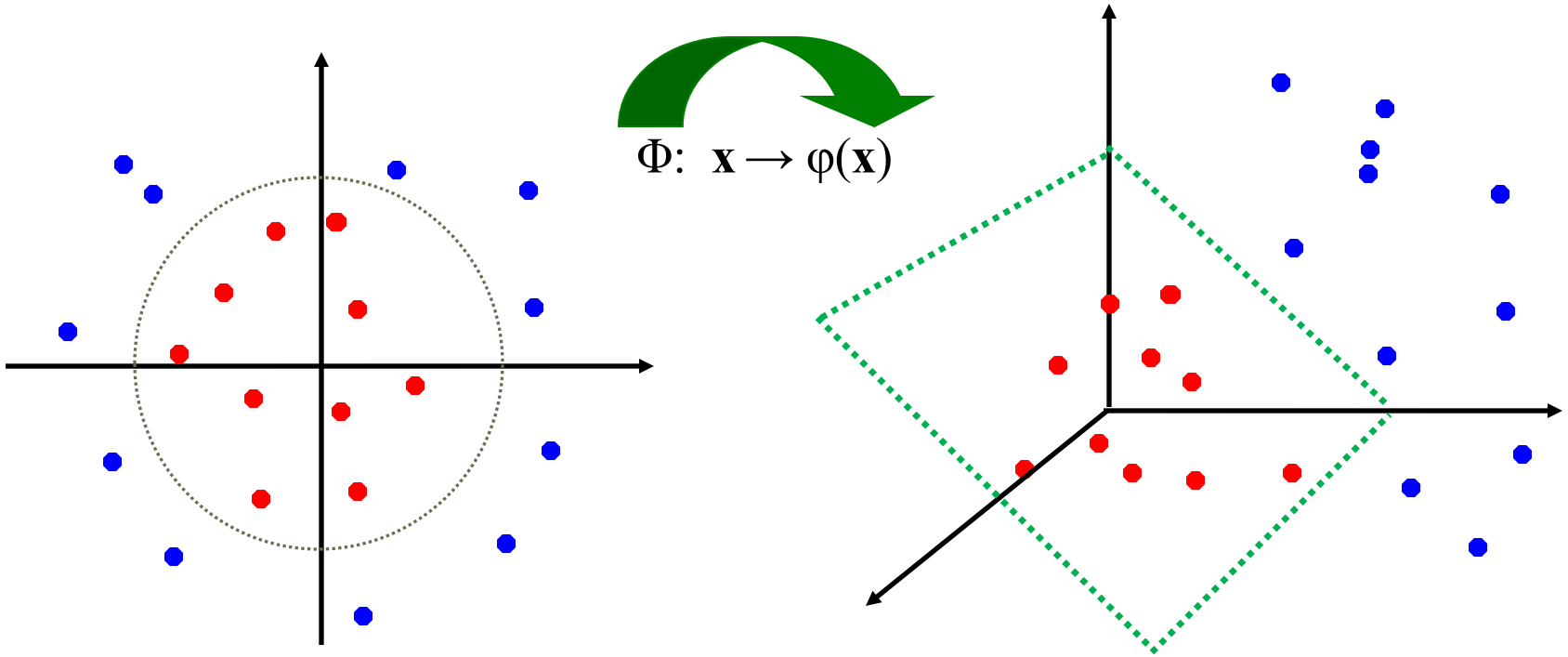
- How about … mapping data to a higher-dimensional space:





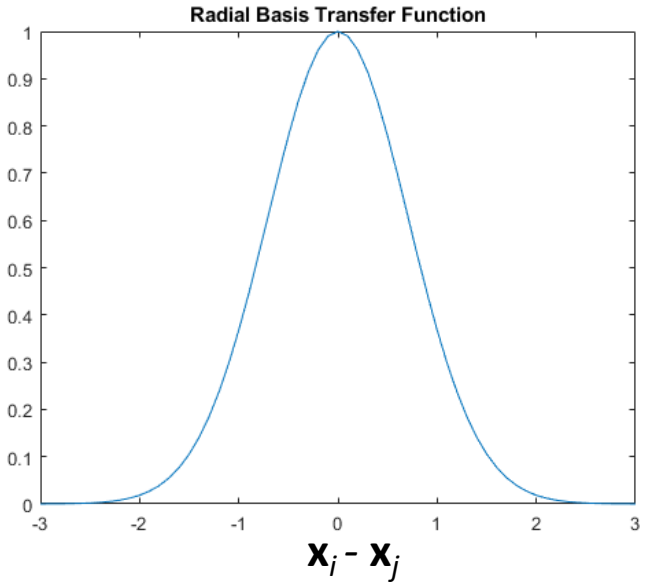
Non-linear SVMs: Feature spaces

- General idea: the original feature space can always be mapped to some higher-dimensional feature space where the training set is separable:



Non-linear SVMs: Feature spaces

- SVM uses a kernel function (which is a similarity measure between data points) to map data from input (feature) space to a higher-dimensional feature space
- Why use kernels?
  - Make non-separable problem separable.
  - Map data into better representational space
- Common kernels
  - Linear $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$
  - Polynomial $K(\mathbf{x}_i, \mathbf{x}_j) = (1 + \mathbf{x}_i^T \mathbf{x}_j)^d$
  - Radial basis function $K(\mathbf{x}_i, \mathbf{x}_j) = e^{-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}} = e^{-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2}$



SVM Reading

- Aurélien Géron. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd Edition, 2022.
  Chapter 5 for a rudimentary introduction.

- Christopher Bishop. Pattern Recognition and Machine Learning. Springer 2006.
  https://www.microsoft.com/en-us/research/uploads/prod/2006/01/Bishop-Pattern-Recognition-and-Machine-Learning-2006.pdf
  Section 7.1 for a more rigorous introduction.

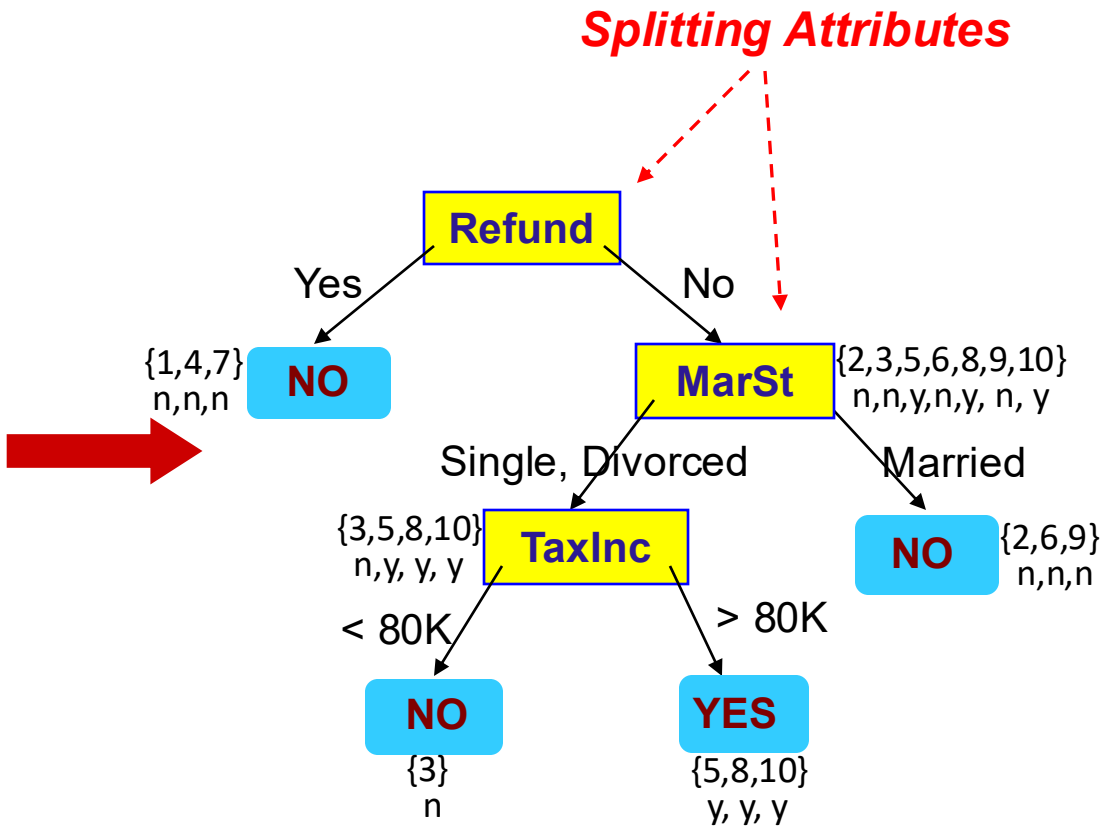
Contents

- k-nearest neighbours
- Support Vector Machines
- **Decision trees**
- Hyperparameter Optimisation

Example of a Decision Tree

	categorical	categorical	continuous	class
Tid	**Refund**	**Marital Status**	**Taxable Income**	**Cheat**
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Training Data



Model: Decision Tree

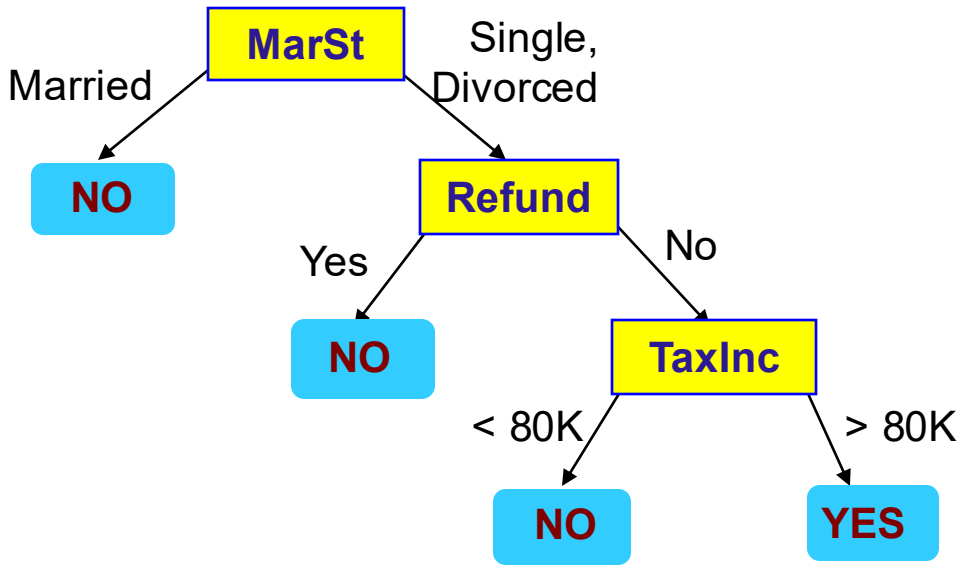
Decision Tree

- A **root node** that has no incoming edges and zero or more outgoing edges
- An **internal node** is a test on an attribute
- A **branch** represents an outcome of the test
- A **leaf** or **terminal node** represents a class label
- At each node, one attribute is chosen to **split** training examples into distinct classes as much as possible

- A new case is classified by following a matching path to a leaf node.

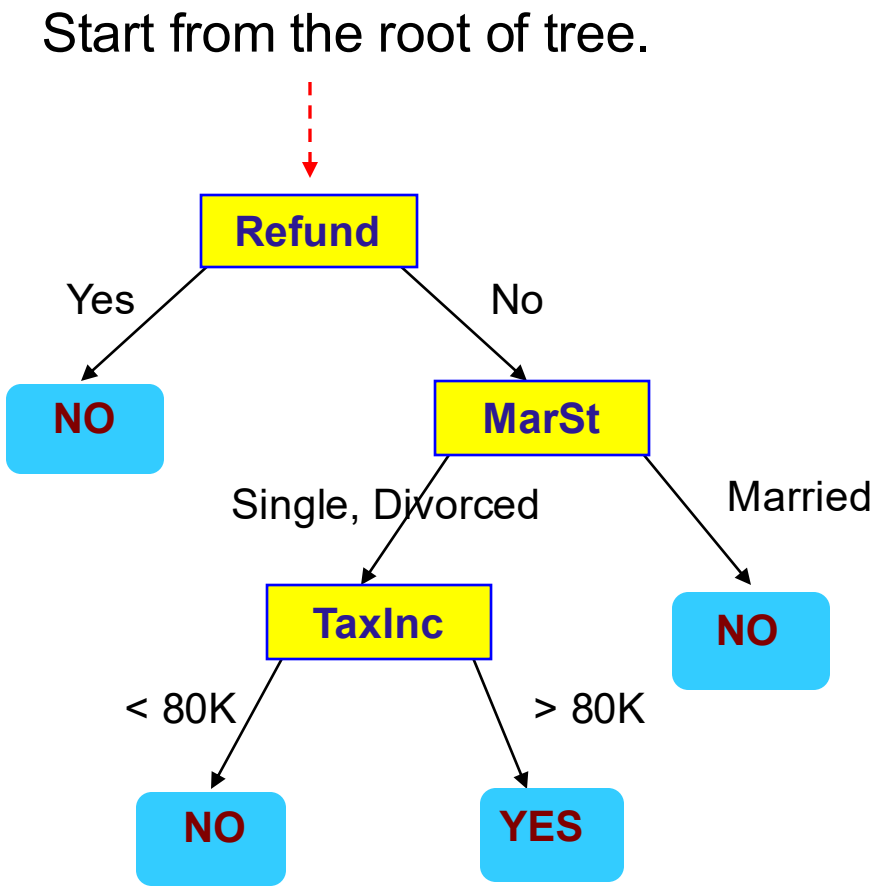
Example of Decision Tree

	categorical	categorical	continuous	class
Tid	**Refund**	**Marital Status**	**Taxable Income**	**Cheat**
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



There could be more than one tree that fits the same data!

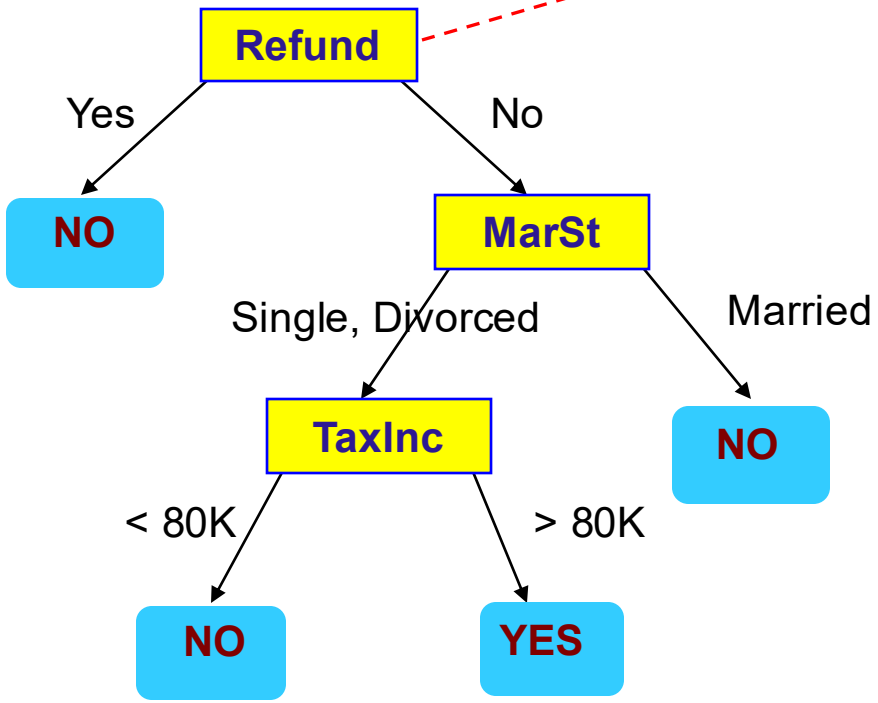
Apply Model to Test Data



Test Data

Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?

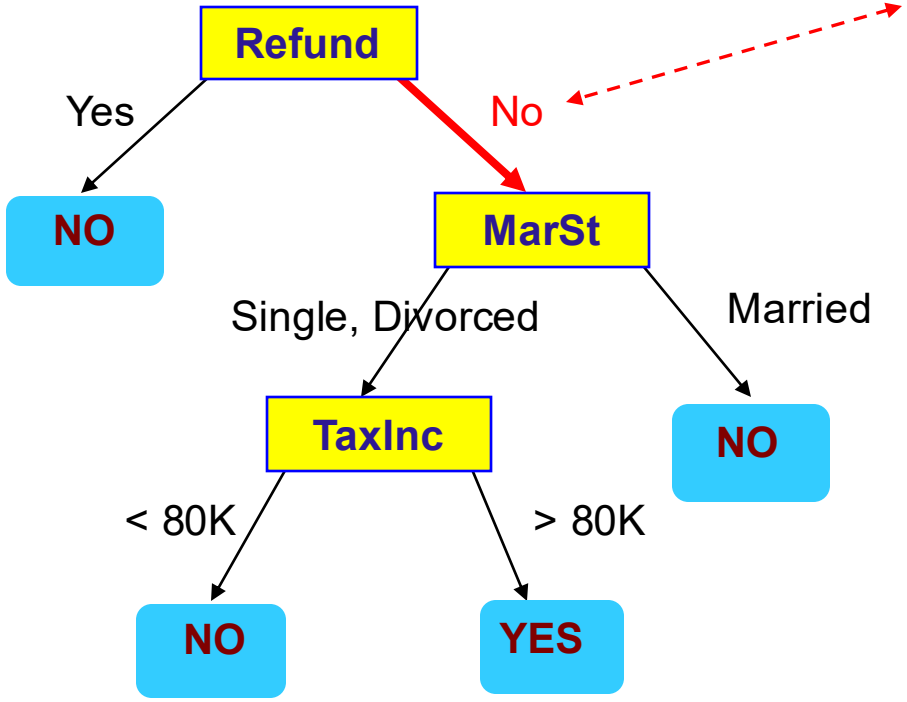
Apply Model to Test Data



Test Data

Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?

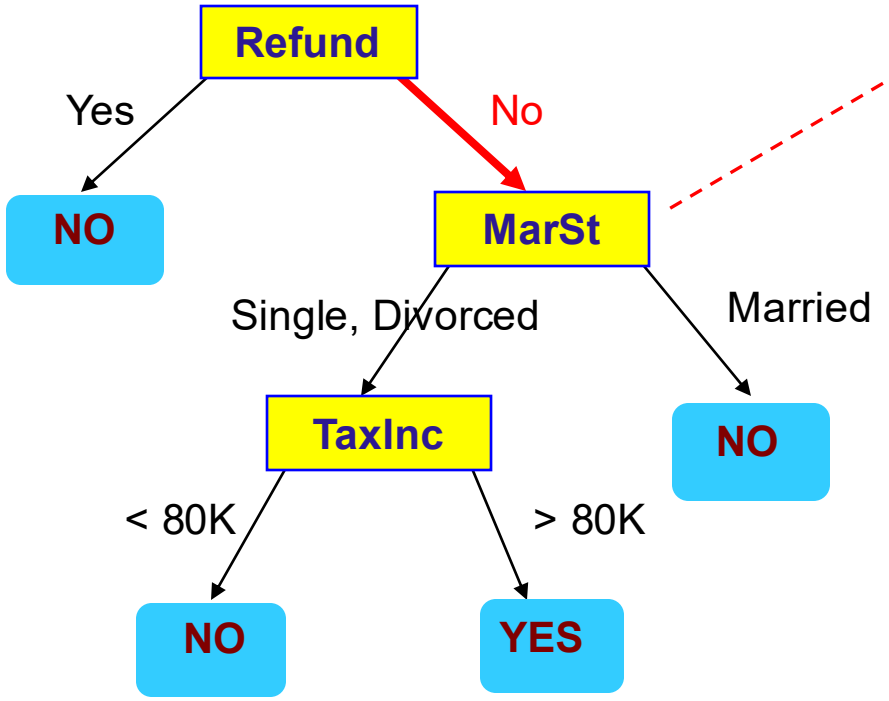
Apply Model to Test Data



Test Data

Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?

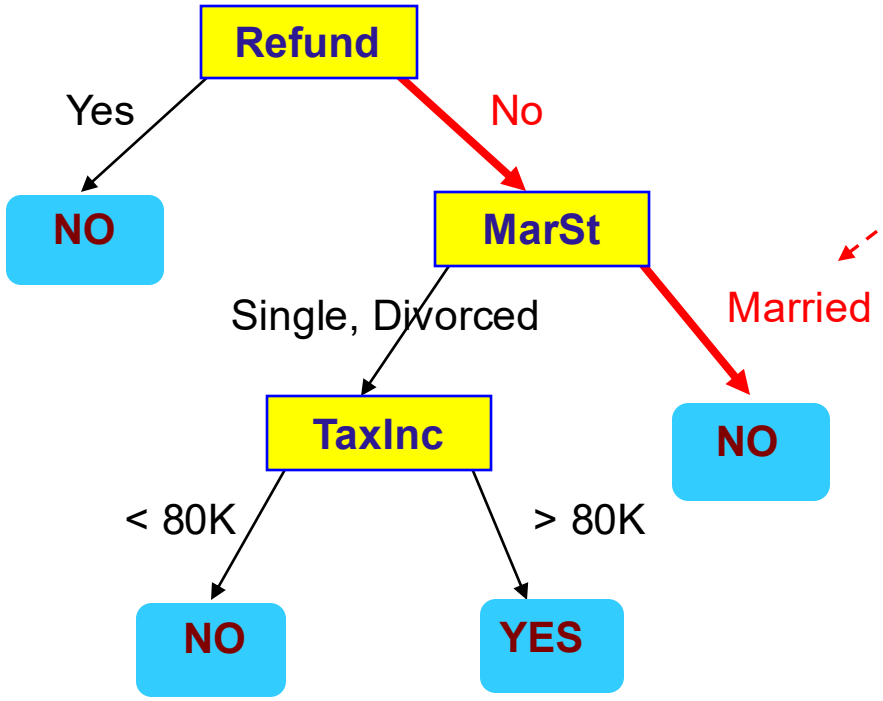
Apply Model to Test Data



Test Data

Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?

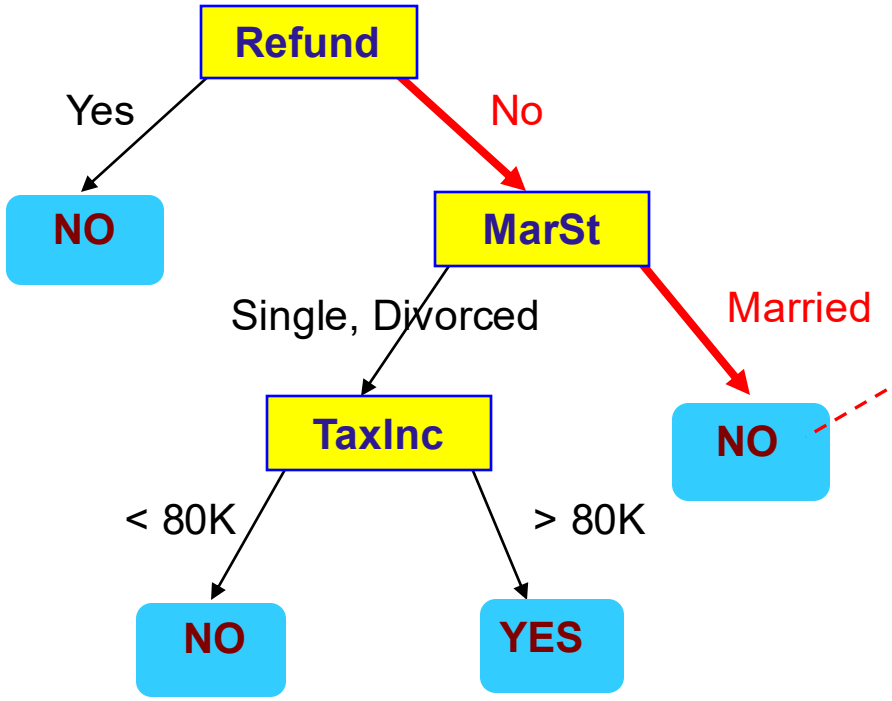
Apply Model to Test Data



Test Data

Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?

Apply Model to Test Data



Test Data

Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?

Assign Cheat to “No”

Decision Tree Algorithms

- Many Algorithms:
  - Hunt's (one of the earliest)
  - CART (Classification And Regression Tree)
  - ID3 (Iterative Dichotomiser 3)
  - C4.5 (successor of ID3)
  - SLIQ (Supervised Learning In Quest), SPRINT

Choosing the Splitting Attribute

- At each node, available attributes are evaluated on the basis of separating the classes of the training examples. A Goodness function is used for this purpose.
- Typical goodness functions:
  - information gain (Entropy) used in (ID3/C4.5)
  - GINI index (IBM Intelligent Miner)

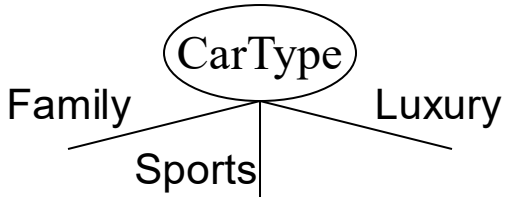
How to Specify Test Condition?

- Depends on attribute types
  - Nominal: takes values from an unordered set. Marital status, Gender, Colour, etc.
  - Continuous: takes values from ordered set. Age, Salary, etc.

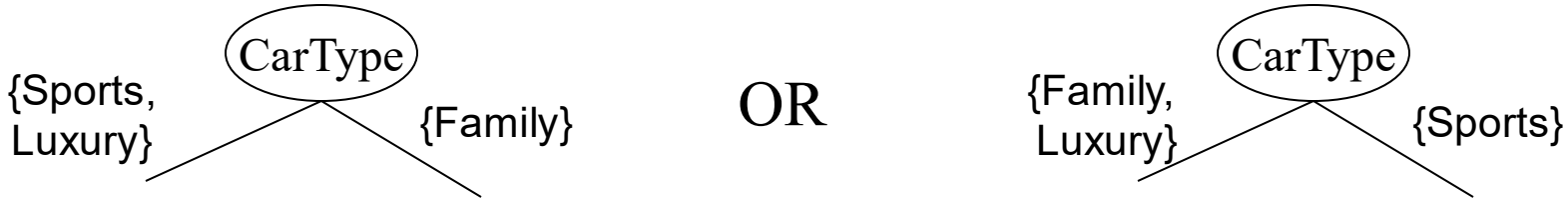
- Depends on number of ways to split
  - 2-way split
  - Multi-way split

Splitting Based on Nominal Attributes

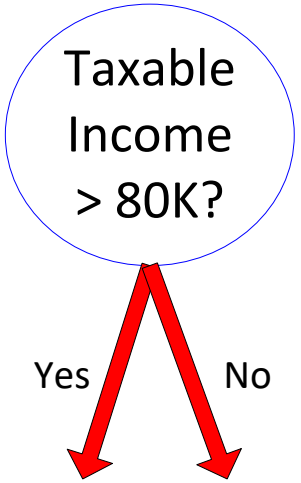
- Multi-way split: Use as many partitions as distinct values.



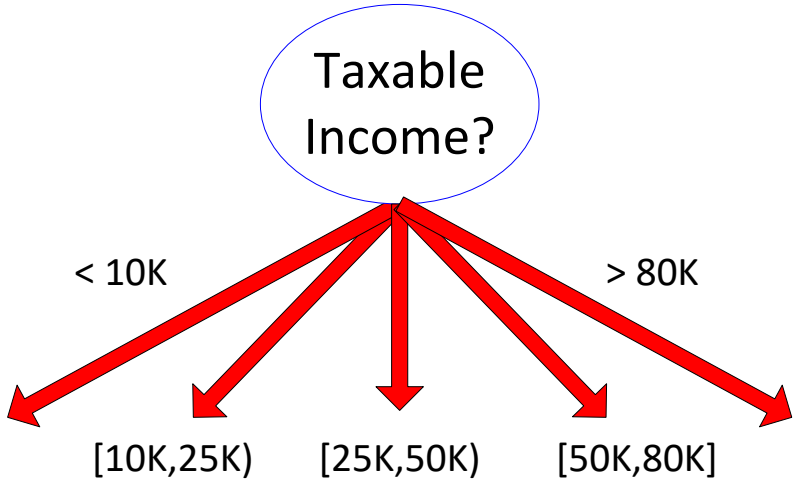
- Binary split: Divides values into two subsets.
  Need to find optimal partitioning.



Splitting Based on Continuous Attributes



(i) Binary split



(ii) Multi-way split

How to determine the Best Split

- Greedy approach:
  - Nodes with homogeneous class distribution are preferred
- Need a measure of node impurity:

C0: 5 C1: 5	C0: 9 C1: 1
Non-homogeneous, **High degree of impurity**	**Homogeneous,** **Low degree of impurity**

A criterion for attribute selection

- Which is the best attribute?
  - The one which will result in the smallest tree
  - Heuristic: choose the attribute that produces the “purest” nodes
- Popular *impurity criterion*: *information gain*
  - Information gain increases with the average purity of the subsets that an attribute produces
- Strategy: choose attribute that results in greatest information gain

Computing information

- Given a probability distribution, the information required to predict an event is the distribution's ***entropy*** *(*Shannon entropy)
- Generally, entropy refers to disorder or uncertainty
- Shannon entropy was introduced by Claude E. Shannon in 1948

Splitting Criteria based on Information Gain

- Entropy at a given node *t*:

$$Entropy(t) = -\sum_j p(j \mid t) \log_2 p(j \mid t)$$

(NOTE: $p(j \mid t)$ is the relative frequency of class *j* at node *t*).

  - Measures homogeneity of a node.
    - Maximum entropy when records are equally distributed among all classes, implying least information
    - Minimum entropy (0.0) when all records belong to one class, implying most information

Examples for computing Entropy

$$Entropy(t) = -\sum_j p(j \mid t) \log_2 p(j \mid t)$$

C1	**0**
C2	**6**

P(C1) = 0/6 = 0 P(C2) = 6/6 = 1

Entropy = – 0 – 1 $\log_2$ 1 = – 0 – 0 = 0

C1	**1**
C2	**5**

P(C1) = 1/6 P(C2) = 5/6

Entropy = – (1/6) $\log_2$ (1/6) – (5/6) $\log_2$ (5/6) = 0.65

C1	**2**
C2	**4**

P(C1) = 2/6 P(C2) = 4/6

Entropy = – (2/6) $\log_2$ (2/6) – (4/6) $\log_2$ (4/6) = 0.92

Splitting Based on Information Gain

- Information Gain:

$$GAIN_{split} = Entropy(p) - \left(\sum_{i=1}^{k} \frac{n_i}{n} Entropy(i) \right)$$

Parent Node, p is split into k partitions;
n_i is number of records in partition i

  - Measures Reduction in Entropy achieved because of the split. Choose the split that achieves most reduction (maximizes GAIN)
  - Used in ID3 and C4.5

Splitting based on *Gini* Index

- If a data set *t* contains examples from *c* classes, gini index, $gini(t)$ is defined as

$$gini(t) = 1 - \sum_{j=1}^{c} p_j^2$$

where p_j is the relative frequency of class j in t.

Example:
a node with 3 classes, each having samples [0, 49, 5]
$gini(T) = 1 - (0/54)^2 - (49/54)^2 - (5/54)^2 \approx 0.168$.

- If a data set t of n points is split into two subsets t_1 and t_2 with sizes n_1 and n_2 respectively, the $gini$ index of the split data contains examples from c classes, the $gini$ index $gini(t)$ is defined as

$$gini_{split}(t) = \frac{n_1}{n}\, gini(t_1) + \frac{n_2}{n}\, gini(t_2)$$

- The attribute provides the smallest $gini_{split}(t)$ is chosen to split the node (*need to enumerate all possible splitting points for each attribute*).

Weather Data: Play or not Play?

- Training Examples

Day	Outlook	Temp.	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Weak	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Strong	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

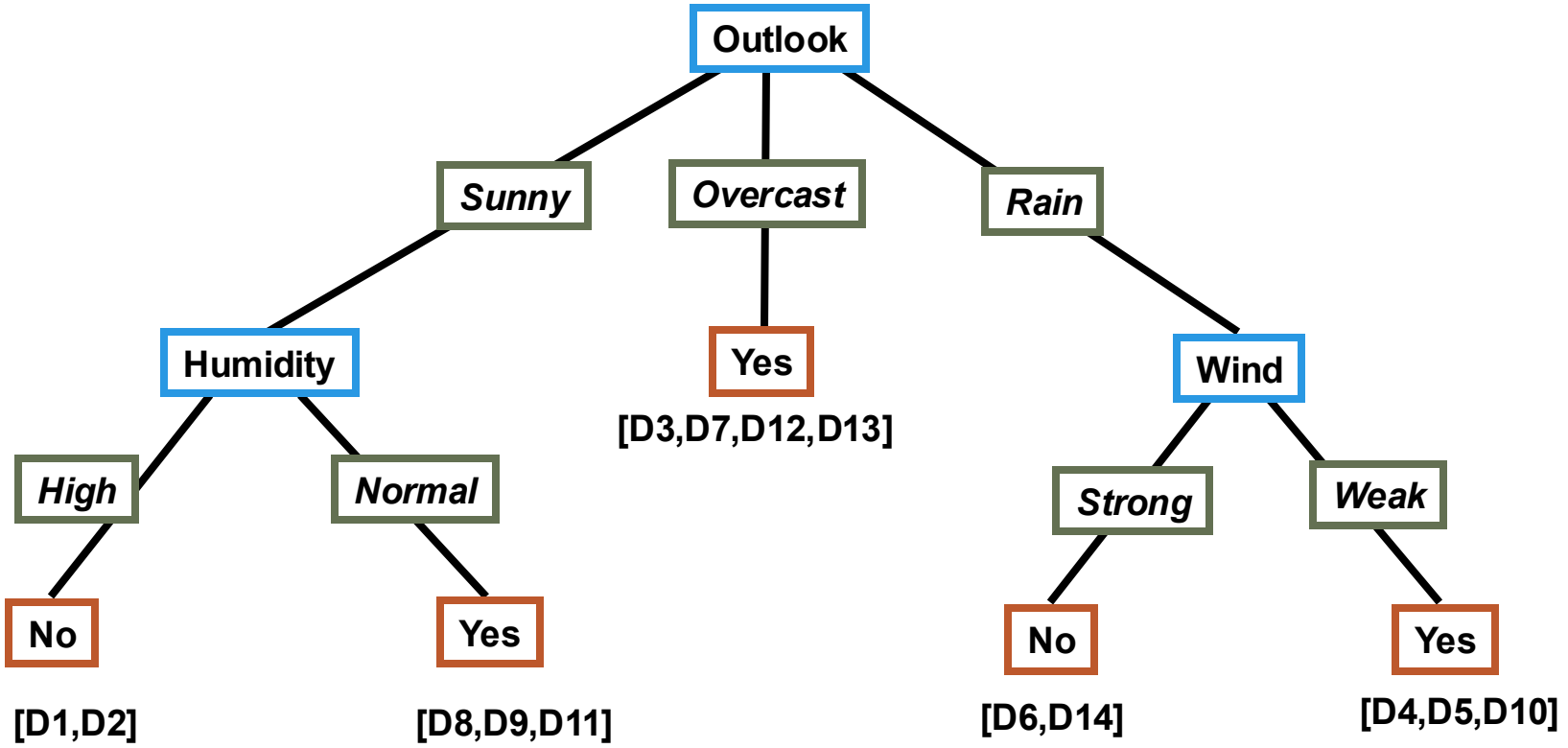
Decision Tree for PlayTennis

- Attributes and their values:
  - Outlook: *Sunny, Overcast, Rain*
  - Temperature: *Hot, Mild, Cool*
  - Humidity: *High, Normal*
  - Wind: *Strong, Weak*

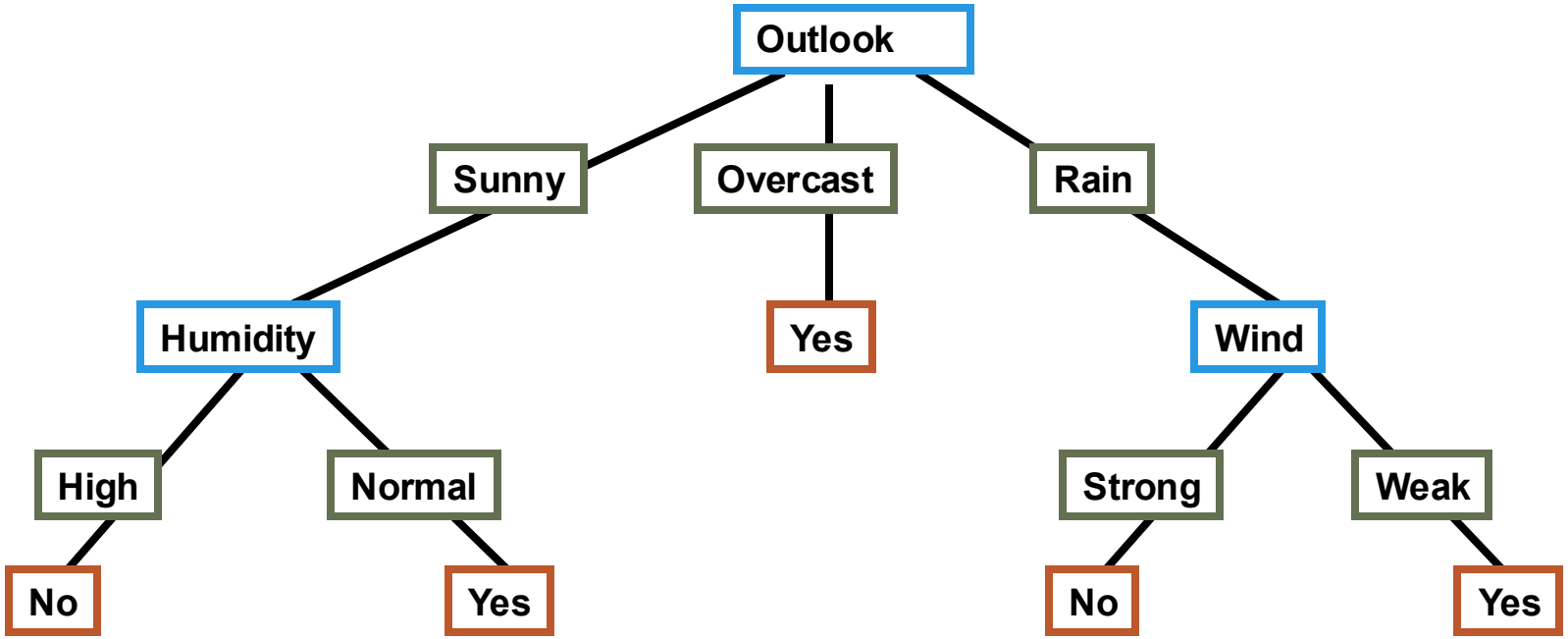
  - Target concept - Play Tennis: *Yes, No*

Day	Outlook	Temp.	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Weak	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Strong	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Decision Tree for PlayTennis from ID3 Algorithm



Converting a Tree to Rules



R_1: If (Outlook=Sunny) $\wedge$ (Humidity=High) Then PlayTennis=No
R_2: If (Outlook=Sunny) $\wedge$ (Humidity=Normal) Then PlayTennis=Yes
R_3: If (Outlook=Overcast) Then PlayTennis=Yes
R_4: If (Outlook=Rain) $\wedge$ (Wind=Strong) Then PlayTennis=No
R_5: If (Outlook=Rain) $\wedge$ (Wind=Weak) Then PlayTennis=Yes

Regression Trees

- Training of regression trees is the same as that of classification tree, except to split the training set in a way that minimizes the MSE
- If a data set (node) t_k of n points is split into two subsets (two nodes) t_1 and t_2 with sizes n_1 and n_2 respectively, then the cost function is

$$J(k, t_k) = \frac{n_1}{n} \text{MSE}_{t_1} + \frac{n_2}{n} \text{MSE}_{t_2}$$

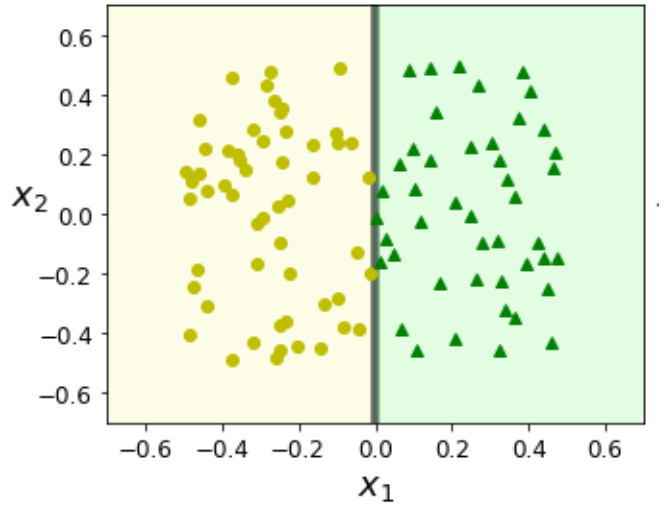
where

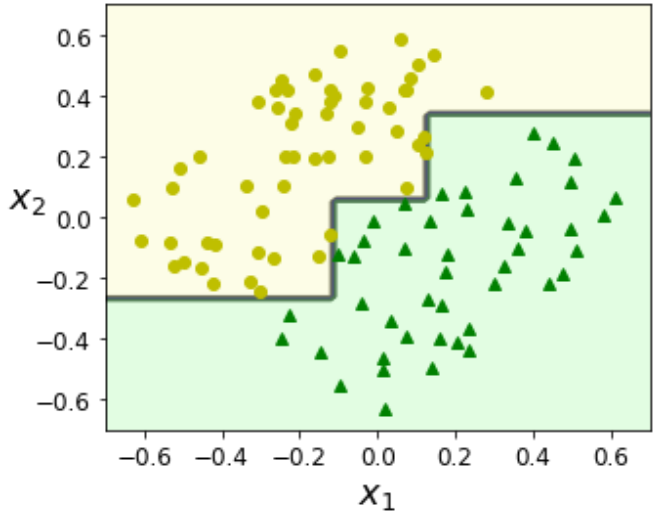
$$\begin{cases} \text{MSE}_{\text{node}} = \sum_{i \in \text{node}} \left(\hat{y}_{\text{node}} - y^{(i)} \right)^2 \\ \hat{y}_{\text{node}} = \frac{1}{N_{\text{node}}} \sum_{i \in \text{node}} y^{(i)} \end{cases}$$

Remarks

- Pros
  - Interpretable decision process: white box model
  - Decision trees requires little data preparation: no need of feature scaling or centering at all

- Cons
  - Splits are perpendicular to an axis, so sensitive to training set rotation
    - Solution: use PCA to get a better orientation of the training data
  - Very sensitive to small variations in the training data, leading to different trees.
    - Solution: use Random Forest to limit instability





Reading

- Aurélien Géron. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd Edition, 2022.
  Chapter 6.

Contents

- *k*-nearest neighbours
- Support Vector Machines
- Decision trees
- **Hyperparameter Optimisation**

Hyperparameter Optimisation

- Model parameters vs. hyperparameters

  SVM Example:

$$f(\boldsymbol{x}_i) = sign(\boldsymbol{w}^T \boldsymbol{x}_i + b)$$

$$\min \quad \frac{1}{2} w^T w + C \sum_{i=1}^{N} \xi_i$$

- Machine learning models have hyperparameters that you must set in order to customize the model to a given dataset,
  - hyperparameter may take on integer-valued, real-valued, or categorical, e.g.,
    - *k* of *k*-NN: integer
    - C and *r* of SVM with RBF kernel: real
    - Kernel of SVM: categorical {'linear', 'poly', 'rbf', …}

Hyperparameter Optimisation

- Machine learning programs usually provide default values for hyperparameters, e.g.,
  - *k*=5 in KNeighborsClassifier of sklearn
  - C=1 and γ= 1 / (n_features * X.var()) in SVC of sklearn

- Hyperparameter optimisation (a.k.a. hyperparameter tuning / search)
  - Searches for a set of hyperparameter values that results in a model that achieves the best performance on a given dataset

Methods of hyperparameter optimization

- Grid search
- Random search
- Bayesian optimization
  - Optuna
  - Hyperopt
  - Scikit Optimize

See https://www.freecodecamp.org/news/hyperparameter-optimization-techniques-machine-learning/

Grid search

- The most basic hyperparameter optimisation method
- Defines a search space as a grid of hyperparameter values and evaluate every position in the grid
- Suffers from the curse of dimensionality since the required number of function evaluations grows exponentially with the dimensionality of the configuration space
- Increasing the resolution of discretization substantially increases the required number of function evaluations

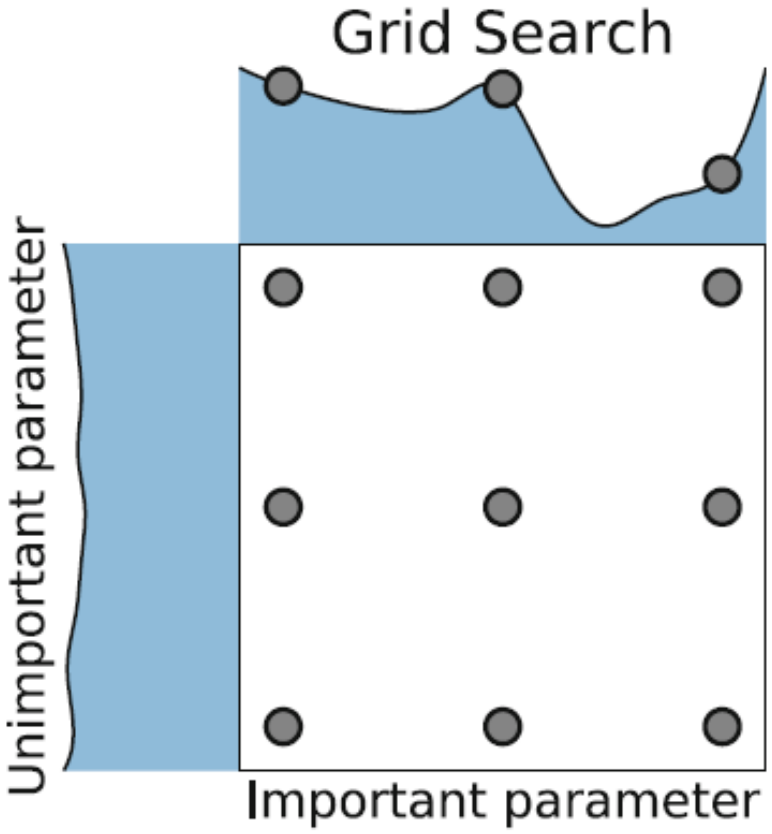


Figure from Feurer & Hutter (2019)
DOI: 10.1007/978-3-030-05318-5_1

Random search

- Define a search space as a bounded domain of hyperparameter values and randomly sample points in that domain
- Works better than grid search when some hyperparameters are much more important than others
- More efficient in terms of exploration. For B evaluations, grid search only explores $B^{\frac{1}{N}}$ different values for each of N hyperparameter, while random search will explore B different values for each hyperparameter
- Easier parallelization (since workers do not need to communicate with each other and failing workers do not leave holes in the design)

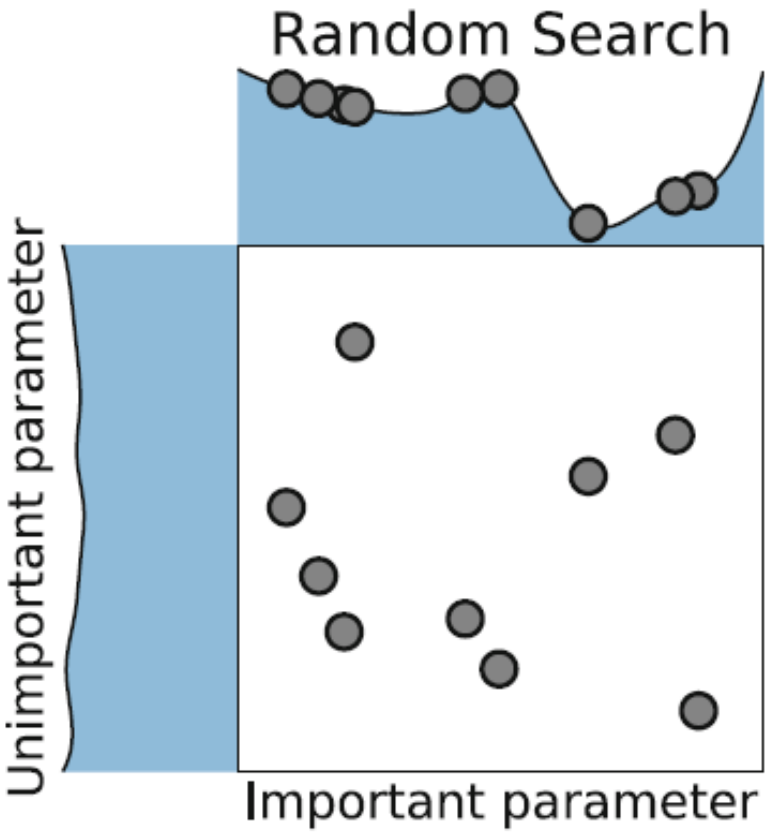


Figure from Feurer & Hutter (2019)
DOI: 10.1007/978-3-030-05318-5_1

Summary

- *k*-NN is a simple but can produce good results for many complex problems
- SVM is an important and popular machine learning method with many successful applications
- DT is a white box method with good interpretability
- All *k*-NN, SVM and DT can be used for both classification and regression